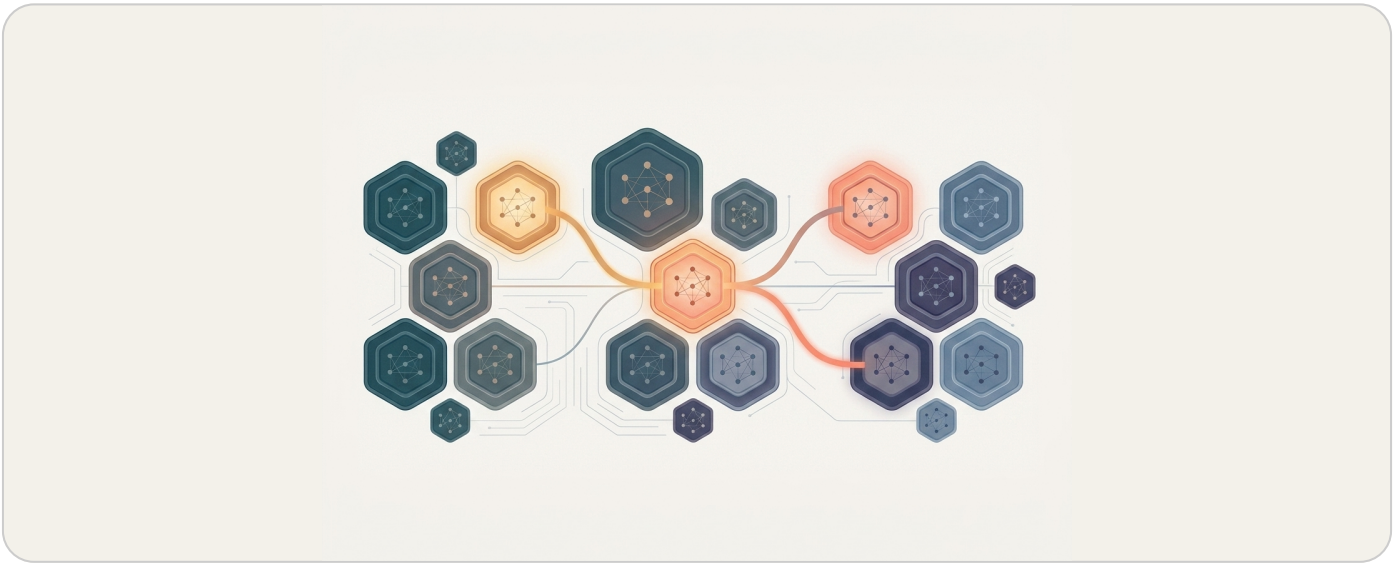
Holonic Neural Networks



What I Built, Why It Mostly Failed, and Why I Still Think the Idea Is Right

Dr. Zachary Welz (Peerless)

2026-07-19

neural architecture | postmortem | knowledge graphs | neurosymbolic | self-organizing systems | neuromodulation

A long postmortem on a neural architecture that treats structure and computation as the same object. I got a reference implementation running, watched it break in instructive ways, patched the breaks, watched it break differently, and eventually concluded that I should write up what the idea was, what actually happened when I tried it, and where I think it's still worth someone's time.

The one-paragraph version

I wanted a neural network that could grow and prune its own structure, forget things it stopped using, refuse inputs that violated its constraints, and be modulated globally by something like arousal or stress (all while remaining inspectable, because the structure would literally be a queryable knowledge graph rather than an opaque weight tensor). I called it a Holonic Neural Network, after Arthur Koestler's "holon" (a thing that is simultaneously a whole and a part). I built a reference implementation on top of an RDF substrate, got it running end to end, and it worked well enough to teach me exactly which of my assumptions were wrong. The short version: the *structural* ideas are sound and, I'd argue, undervalued. The *learning* story is where it falls down, and it falls down for reasons that are somewhat fundamental and somewhat just-needs-more-engineering. This is my writeup.

1. How the central idea formed

Modern deep learning runs on a very specific substrate: dense tensor operations over a topology that is fixed at design time, trained by propagating gradients backward from a global loss. This has worked spectacularly. I'm not here to argue it hasn't. But if you stare at it with the right kind of dissatisfaction, a few things stand out. They aren't obvious limitations; they're *commitments* that were made early and never revisited:

1. **The topology is frozen.** A trained network cannot grow a new region because it encountered a new kind of problem, or prune a dead one, or rewire itself. Architecture search happens *outside* the network, by humans or by an outer optimization loop. The network itself has no say in its own shape.
2. **Every connection is a scalar.** A weight says *how much* signal flows, never *what kind* is allowed. There is no notion of a connection having a contract, no way to say "I only accept this shape of input, and I'll refuse anything else."
3. **Message passing is homogeneous.** Every node-to-node interaction is the same operation (matmul, nonlinearity), regardless of what the two nodes represent. A biological brain does not use the same computation for edge detection in V1 and value estimation in the striatum.
4. **There's no global modulatory state.** In a brain, neuromodulators (dopamine, norepinephrine, acetylcholine, serotonin, cortisol) rescale the dynamics of whole circuits at once. They change learning rates, gate plasticity, shift exploration versus exploitation. A transformer has, at inference, exactly one global scalar: sampling temperature, and it only touches the output distribution.
5. **There's no memory of use.** The network doesn't know which of its parts are hot and which are cold. Forgetting is either impossible (frozen weights) or catastrophic (the continual-learning failure mode, where new learning stomps on old).
6. **The structure is not legible.** You cannot ask a trained network "what do you know about X and how is it connected to Y?" and get a structural answer. That knowledge is lost across millions of latent parameters. Interpretability is an entire research field precisely *because* the substrate made no provision for it.

Biological neural systems have none of these limitations. Cortex grows, prunes, and rewires across the lifespan. Synapses gate conditionally. Different areas compute differently. Neuromodulators rescale globally. Used circuits consolidate; unused ones atrophy. And the structure is legible at every scale we can measure, from cortical columns to functional areas to whole-brain networks. Artificial neural networks were of course inspired by these systems, but the resemblance is shallow: we borrowed the weighted-sum-and-nonlinearity caricature of a neuron and almost none of the architectural properties above. There has been real work on individual pieces (developmental and neuromodulated approaches, growing and pruning networks, continual learning) but comparatively little on recovering them together, and the mainstream substrate has drifted further from biology, not closer, as it scaled.

So the question I couldn't put down was: **what would a substrate look like that recovered these properties, without throwing away what deep learning got right about distributed representation and differentiable optimization?**

The Holonic Neural Network was my initial attempt at an answer. The central bet is a single sentence: **structure and computation should be two views of the same object.** Not a neural net *connected to* a knowledge graph. One thing, looked at two ways.

A quick note on what this paper is. It is not a benchmark report; the empirical content simply is "I built it and here's how it broke," not "it achieves X on Y." It's also not a pure vision paper, because I did build the thing and the building taught me most of what's worth sharing. Treat it as a design document with a postmortem stapled to it.

---

2. The holon, and where it comes from

The word "holon" is Arthur Koestler's, from *The Ghost in the Machine* (1967). He built it from the Greek *holos* (whole) plus the suffix *-on* (part, as in proton), to name something that is simultaneously a self-contained whole and a component of something larger. Your hand is a whole (it has its own integrity, its own subsystems) and a part (of your arm, of you). Koestler called the nesting of holons a **holarchy**, and he pointed at what he called the "Janus effect": every holon faces two directions at once, asserting itself as a whole while integrating as a part.

Two historical footnotes worth having, because they cut both ways.

First, the holon concept has real engineering pedigree. It didn't stay philosophical. Holonic Manufacturing Systems took it seriously in the 1990s; the PROSA reference architecture (Van Brussel et al., 1998) defines Product, Resource, Order, and Staff holons and is still cited. More recently Kurt Cagle has carried the holon into graph and RDF architecture, and his four-graph decomposition (below) is what gave me something concrete to build against. So "holon as a unit of a self-organizing system" is not something I invented; it's a lineage I'm borrowing from.

Second, and this needs stating: Koestler, in the *same book*, leaned on the triune-brain model (the idea that we have a reptilian core, a paleomammalian limbic layer, and a neomammalian neocortex stacked like geological strata). That model is now considered wrong by comparative neuroscientists (more on this in the section I got wrong and fixed, because I made the same mistake in my first draft). The holon concept stands on its own; the neuroscience it was bundled with does not.

The reason the holon is the right primitive for what I wanted is that it already carries the two properties I most needed: **boundedness** (a holon has an inside and an outside, and something mediates between them) and **nesting** (holons contain holons, giving you scale without a separate mechanism). A neuron has neither. A layer has the first but not really the second. A holon has both, natively.

2.1 The four-graph model, and Cagle's version of it

Here's where the implementation substrate came from. Kurt Cagle has been developing what he calls a Holonic Graph Architecture on top of the RDF stack, and it gave me a concrete, inspectable way to represent a holon. In his four-graph decomposition, each holon is not a single blob but four layered named graphs:

- an **interior**: the holon's actual contents, its knowledge, as a graph of triples;
- a **boundary / membrane**: constraints (expressed as SHACL shapes) that define what is allowed to be inside, and by extension what may cross in;

- **portals**: governed interfaces to other holons, expressed as queries (SPARQL CONSTRUCT) that transform and forward content;
- **provenance**: the activity trail (PROV-O) of what happened to the holon and when.

In pseudo-code, the holon I actually implemented was this (four graph layers, plus the one field the four-graph model doesn't have, energy, that the HNN adds):

```
Holon:
    interior    : Graph          # contents: triples, or a learned embedding
    membrane    : ShapeSet       # SHACL shapes; what is allowed inside
    portals     : [Portal]       # typed, governed outgoing interfaces
    provenance  : [Event]        # append-only activity trail
    energy      : float          # HNN addition: scalar activation / "warmth"

Portal:                          # the HNN analog of a synapse, but typed
    target      : HolonRef
    transform   : Signal -> Signal   # CONSTRUCT query or learned function
    weight      : float          # scalar multiplier, updated by learning
    resistance  : float          # traversal cost, derived from transform complexity
    type_sig    : (InShape, OutShape)   # contract on what may cross

# admission is rejection-first, not attenuation:
def receive(holon, signal):
    if not holon.membrane.validates(signal):
        provenance.record("membrane_reject", signal)   # a first-class outcome
        return REJECTED                                # refused, not absorbed
    holon.interior.integrate(signal)
    holon.provenance.record("interior_update", signal)
```

The thing to notice is `receive`. In a normal net, an invalid or unexpected input is just aggregated in with everything else and attenuated. Here, rejection is a first-class outcome; the membrane can *refuse*, and the refusal is recorded. That single design choice is where contradiction detection later comes to live.

What sold me on this decomposition is a move Cagle makes that I think is clever: he maps the boundary layer onto Karl Friston's **Markov blanket** (the statistical boundary that separates a system's internal states from external ones) and reframes SHACL constraint validation as *prediction-error measurement*. A constraint violation isn't just "invalid data." It's a signal that the holon's model of what-belongs-here has diverged from what it's actually receiving. That reframing turns a static validation mechanism into something that looks a lot like the error signal in predictive processing. I'll come back to whether that reframing pays off (it half does).

Two notes about this substrate. Cagle's work is an emerging architecture, developed largely in public writing and a W3C community group rather than as a peer-reviewed standard; I'm citing it as a live idea I built on, not as settled ground. And standard SHACL validates a *single* graph, not a dataset of named graphs, which matters if every holon is a named graph; there's recent work (SHACL-DS) on exactly this gap, and a production system would need it.

The critical thing, though: **the RDF four-graph model is a reference implementation, not a requirement.** Any substrate that gives you (a) bounded units with addressable interior state, (b) typed, governed connections, (c) containment, and (d) queryable introspection can host the framework. RDF happens to give you all four for free and makes the whole system inspectable via SPARQL, which is worth a great deal when you're debugging. But the interior of a holon could just as well be a tensor, and in my implementation, it eventually was.

The anatomy of a single holon looks like this:

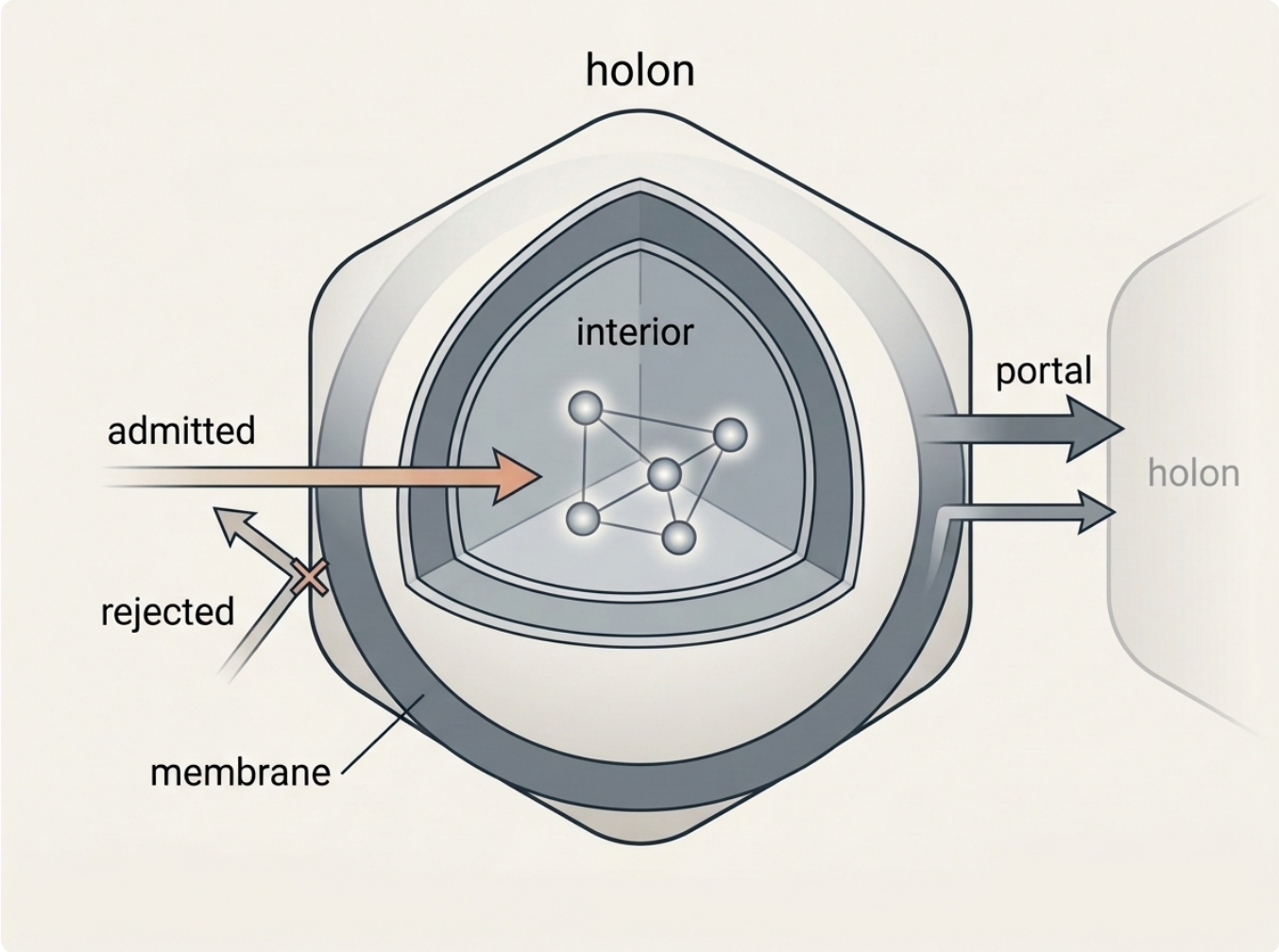


Figure 1. Anatomy of a holon: an interior graph of contents, a membrane that admits or rejects incoming signals, typed portals to other holons, and (tracked but not drawn here) the provenance trail and scalar energy state the text describes.

3. How this differs from things you already know

I want to be precise about the comparisons, because the fastest way to dismiss this framework is to match it against something familiar and move on. So here is the historical literature research I did against the obvious neighbors, and where each of them diverges.

Transformers. A transformer is, structurally, a fully-connected message-passing network over a sequence, with learned attention deciding which messages matter. It has no built-in containment, no locality, no specialization; everything is learned from data, which is exactly why it's so general and so data-hungry. Its topology is fixed and dense. Its connections (attention weights) carry no contract. It has no modulatory state touching internal dynamics. An HNN can put a transformer *inside* a holon (a holon's interior computation can be anything), but the HNN's organizing commitments sit above it. The relationship is simply containment and cannot reflect internal or cross-system competition.

Graph neural networks. GNNs are the closest surface match: they compute over graphs by message passing. I have worked with GNNs a fair amount and find them extremely valuable, but they are not directly applicable to the problem I was trying to solve here. GNN nodes are homogeneous, edges carry scalar or vector weights rather than typed transformations with contracts, message passing is uniform across the graph, and the topology is fixed per training run. There's no membrane; an invalid message just gets aggregated in with everything else. An HNN is not "a GNN with extra features." The organizing commitment is inverted: in a GNN, the graph is a dataset that computation runs *over*; in an HNN, the graph is just the computation.

Neurosymbolic systems. This is the comparison I care most about, because it's where the framework wants to live. The dominant neurosymbolic paradigm connects a neural component and a symbolic component across an interface: neural front-end produces symbols, symbolic back-end reasons over them, or logic constraints regularize a neural loss. Henry Kautz's much-cited taxonomy (six types, from a 2020 talk, and worth knowing it was never a formal paper, just an influential framing) organizes these by *how the two components talk to each other*. The HNN's bet is different: don't build an interface, dissolve the boundary. A holon's interior can hold symbolic triples *and* a learned embedding at once, with the membrane mediating between them. There's no pipeline seam because there's no pipeline. I ended up calling this **neural-structural** rather than neurosymbolic, to mark the difference: the structure isn't a component the neural part talks to, it serves as the substrate the computation happens in. Whether that distinction earns its own word or is just the far end of the neurosymbolic spectrum is a fair thing to argue about.

Mixture-of-experts. MoE is the closest *mainstream* idea to holonic specialization: route each input to specialized sub-networks. Modern MoE models are enormous and sparse (Mixtral 8x7B has 46.7B parameters but activates only ~12.9B per token). But MoE routing is a learned soft gate with no structural contract, experts are usually homogeneous, there's no containment hierarchy, and there's no membrane. You can read MoE as a degenerate HNN: a two-level holarchy (router plus experts), membranes removed, portals unconstrained. I will admit I am ignorant of some of the deeper engineering details of how modern MoE systems are actually built, and that gap is mine to close. What I have not been able to see is how MoE routing addresses the specific problems I was focused on: structural mutability, connections that carry contracts, and a topology you can inspect.

The summary of this section: none of the HNN's individual commitments are novel. Governed connections exist in typed systems. Self-organization exists in developmental and evolutionary methods. Neuromodulation exists in neuromodulated RL. Containment exists in hierarchical models. The bet is that **unifying all of them under one primitive** buys something the pieces don't buy separately. Whether that bet pays off is exactly what building it was supposed to test.

4. Energy is the first part that has to work

Before any learning, you need dynamics, a story for how activation moves through the holarchy. This is the part I most underestimated, and it's where my first implementation face-planted, so I'm going to be concrete.

4.1 The idea

A stimulus enters at a holon whose membrane admits it (a sensory entry point). That holon's energy rises. Energy then propagates outward across portals and up/down through containment, attenuating with distance:

```
a(target) = a(source) · exp(−λ · d(source, target))
```

where `λ` is a global decay constant (itself modulated, see Neuromodulation) and `d` is a composite distance (part portal-path length, part containment depth, part embedding similarity). Frequently-activated holons stay "warm" (a rejuvenation term lifts their baseline); holons that stop being activated cool off and decay. Cold holons become candidates for pruning, merging, or archival. Hot regions correspond to knowledge in active use. That energy landscape, which parts are warm and which are cold, is the thing structural learning acts on.

This is Hebbian consolidation lifted from the level of individual synapses to the level of whole subgraphs, and it's the mechanism that's supposed to give the system a *memory of use*.

4.2 The results of my experimentation

It oscillated, saturated, or died. Repeatedly. Let me give you the three most instructive failures, because they're more useful than the design intent.

Failure one: energy that couldn't decay. My first propagation rule gave every holon that received a portal firing a flat additive energy bump. Sounds fine. But in a densely connected little holarchy, every holon receives firings from several neighbors every tick, and the flat bumps summed faster than the decay term could remove them. The whole network pinned to maximum energy and stayed there. Every holon was maximally "hot," which is the same as no holon being hot; the signal that was supposed to distinguish used from unused knowledge carried zero information. The equilibrium energy worked out to exactly `1.0 · (1 − decay_rate)` regardless of input, which is a very tidy way of saying "the input stopped mattering."

Failure two: energy that couldn't propagate. I fixed failure one by making the energy contribution proportional to signal strength rather than a flat bump. Overcorrected. Now the per-firing contribution was so small it never cleared the activation threshold at the next holon, so a stimulus injected at an entry point died one hop in. Nothing downstream ever lit up. I had traded a network that was uniformly on for one that was effectively off.

Failure three: the gate was set for the wrong scale. The activation threshold was gated by the arousal modulator, and at the small scale I was testing (eight holons), arousal pinned high, which pinned the threshold high, which meant a single portal firing could never trigger a cascade; you needed two or more simultaneous firings into the same holon. So cascades only happened by coincidence. The fix that finally worked was a redesign: portal firings contribute *energy floors* that are applied **after** decay (so decay always runs and can

never be outrun), floors sum across multiple simultaneous firings but cap below the direct-injection maximum (so a holon lit only by propagation is always at least slightly cooler than one lit by real input), and the arousal gate coefficient was retuned for the actual holon count.

Here's the propagation step that finally worked, written out, because the *ordering* is the whole point. Decay runs before the floors are applied, so decay can never be outrun; floors are capped below the direct-injection maximum, so a holon lit only by propagation is always at least slightly cooler than one lit by real input:

```
def tick(holarchy):
    floors = {}                                  # target_id -> energy floor

    # 1. propagate from every "hot" holon along its portals
    for src in holarchy.holons:
        if src.energy <= threshold(src):         # threshold is arousal-gated
            continue
        for portal in src.portals:
            signal = portal.transform(src.state) * portal.weight
            signal *= exp(-lambda_ * portal.resistance)   # attenuate with distance
            tgt = portal.target
            tgt.pending += signal
            # floors SUM across simultaneous firings, but CAP below 1.0
            contribution = min(SIGNAL_CAP, norm(signal))  # SIGNAL_CAP < 1.0
            floors[tgt] = min(FLOOR_CAP, floors.get(tgt, 0) + contribution)

    # 2. decay ALWAYS runs first: this is the fix for "energy can't decay"
    for h in holarchy.holons:
        h.energy *= (1 - h.region.decay_rate)

    # 3. THEN apply floors: a holon that received signal is lifted to at
    #    least its floor, but direct injection (below) can still exceed it
    for h, floor in floors.items():
        h.energy = max(h.energy, floor)

    # 4. direct sensory injection is the only thing that reaches energy == 1.0
    for h, stimulus in inbox():
        if h.membrane.validates(stimulus):
            h.energy = 1.0
            h.integrate(stimulus)
```

The three failures above map cleanly onto three lines of this. Failure one (energy can't decay) was applying floors *additively before* decay, so step 3 ran before step 2 and the bumps outran the multiplier. Failure two (energy can't propagate) was `SIGNAL_CAP` set so low nothing cleared `threshold()`. Failure three (wrong-scale gate) was `threshold()` returning a value tuned for four holons while I was running eight.

The reason I'm dwelling on this: **the dynamics are load-bearing and they are not "obvious once you write them down."** Every constant (decay rate, floor coefficient, cap, gate factor) interacts with the topology and the scale in ways that produced qualitatively different global behavior. This is not a domain where you specify the elegant equation and move on. It's closer to tuning a physical system. I lost more time here than anywhere else, and I don't think I was being especially dumb; I think the problem is genuinely finicky.

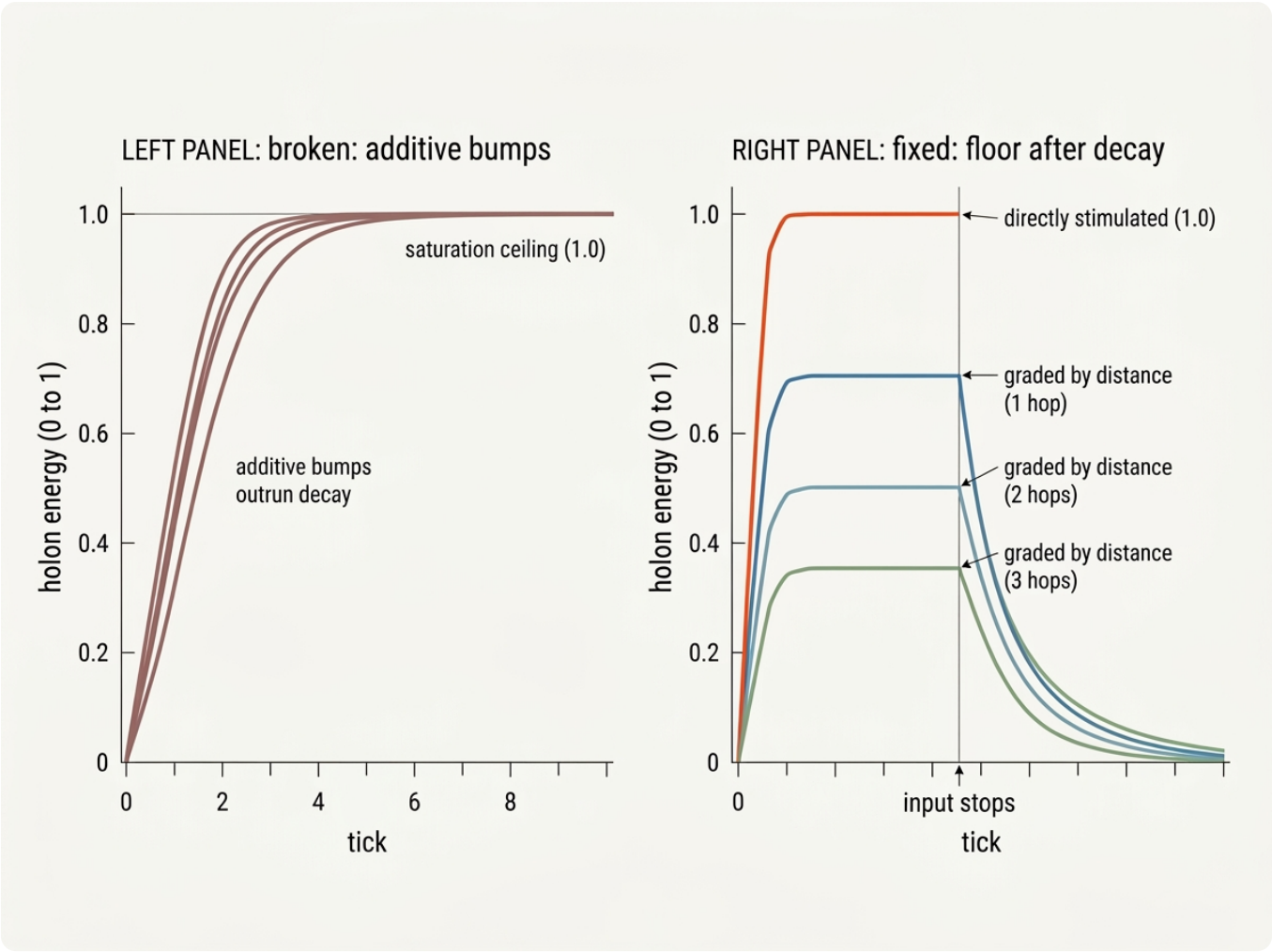


Figure 2. Energy over time. Left: the broken additive rule, where bumps outrun decay and every holon saturates at the ceiling. Right: the floor-after-decay fix, where a directly-stimulated holon holds at 1.0, propagation-lit holons sit at plateaus graded by distance, and all of them decay once input stops.

5. Neuromodulation as global scalars that rescale everything

This is the part that interests me most, because it is the part actually grounded in neuroscience. My wife is in nursing, and over the years I have read a lot of the philosophical and psychological writing that neuroscientists produce, so this is the corner of the design where I had some footing rather than just analogy.

In brains, neuromodulators are chemically distinct from the neurotransmitters that carry point-to-point signal. Glutamate and GABA carry information; dopamine, norepinephrine, serotonin, acetylcholine, and cortisol *rescale the dynamics of information processing*. They change learning rates, shift attention thresholds, tune exploration versus exploitation, mount stress responses. They're broadcast fields, not wires. The computational-neuroscience literature formalizes a good chunk of this through **three-factor learning rules**: a Hebbian eligibility trace (a synapse flags that it was recently co-active) times a *third factor* (a neuromodulator signaling reward,

novelty, or surprise) that decides whether and how the flagged synapse actually changes. That's a real, well-supported framework (Frémaux & Gerstner; Gerstner et al. 2018), and it maps almost one-to-one onto what I wanted.

The HNN carries a handful of global scalars, each named for a biological analog but not exactly that thing, hence the "-ish":

- **Arousal** (norepinephrine-ish): raises the decay constant λ, narrowing the activation radius. High arousal = focused, deep processing over few holons. Low arousal = broad, shallow spread.
- **Reward** (dopamine-ish): when a portal firing contributes to a good outcome, the reward signal reinforces that portal's weight. This is the third factor. It's local credit assignment at the portal rather than backprop through the whole graph, which is precisely what makes it compatible with a topology that's changing underneath you.
- **Stress** (cortisol-ish): repeated membrane rejections, circular provenance, or energy-budget violations raise stress, which tightens membranes and reduces throughput. It's an immune response; the system trades exploration for structural self-protection.
- **Novelty**: newly created holons and portals get a temporary energy bonus so fresh structure gets explored instead of ignored. Novelty against reward is the exploration/exploitation knob.
- **Attention gain** (acetylcholine-ish): sharpens membrane selectivity without narrowing the activation radius, separating "what gets admitted" from "how far things spread."

Portal weight learning is a three-factor rule almost verbatim: an eligibility trace (which portals just fired) times the global reward signal, with a negativity bias so bad outcomes bite harder than good ones reward:

```
# each portal keeps a decaying eligibility trace of recent firing
def on_portal_fire(portal):
    portal.eligibility = 1.0

def decay_traces(portals):
    for p in portals:
        p.eligibility *= TRACE_DECAY          # ~1s-equivalent window

# the third factor gates whether the flagged synapse actually changes
def apply_reward(portals, reward):            # reward in [-1, +1]
    gain = REWARD_LR if reward > 0 else REWARD_LR * NEGATIVITY_BIAS
    for p in portals:
        p.weight += gain * reward * p.eligibility
        p.weight = max(WEIGHT_FLOOR, p.weight)   # never let weights hit zero
```

`WEIGHT_FLOOR` and `NEGATIVITY_BIAS` are both there because of bugs rather than theory (see below).

The design commitment I care about: **these scalars are produced by holons, not set by an operator.** A stress holon watches structural integrity and emits the stress signal. A reward holon watches objective satisfaction. The modulator field is a product of the system's own state, which is what distinguishes it from a hyperparameter schedule. In principle. In practice this is also where I have the least evidence it does anything useful, because you only see the payoff at a scale I never reached.

What *did* work, and pleasantly: reward-modulated portal reinforcement with a proper negativity bias (bad outcomes weighted heavier than good ones, as in real learning) produced sensible, stable weight changes once I fixed the reward *signal* itself. That brings me to the single most embarrassing bug in the project.

5.1 The reward-sign bug, as a cautionary tale

My reward signal was `−tanh(loss)`. Looks reasonable: low loss, signal near zero; high loss, signal near −1. The problem is that for cross-entropy loss, which is always positive, `−tanh(loss)` is *always negative*:

```
# BROKEN: cross-entropy loss is always > 0, so this is always < 0.
# every single training step delivers a punishment. no input can earn a reward.
reward = -tanh(loss)          # loss > 0  ->  reward in (-1, 0)  ->  always negative

# with apply_reward() above, weights only ever decrease. they crash toward
# WEIGHT_FLOOR, the output detaches from the input, and the net collapses to
# "predict whatever the bias says": which turned out to be one token, always.
```

There was no configuration of inputs that produced a positive reward. Portal weights therefore decayed monotonically toward the floor, the network's output detached from its input entirely, and (my favorite detail) it converged on always predicting the same token regardless of what you asked it, because with all portal weights crushed flat, the output was just whatever the bias terms said.

The fix was to make reward *accuracy-based*: correct prediction earns a positive signal proportional to confidence; wrong prediction earns a negative one proportional to how wrong:

```
# FIXED: reward is signed by correctness, so good outcomes actually reward.
def reward_signal(prediction, target):
    if prediction.argmax == target:
        return +prediction.confidence          # in (0, +1]
    else:
        return -min(1.0, prediction.error)     # in [-1, 0)
```

Both are obvious in hindsight. Neither was obvious while I was staring at a network that had confidently decided the answer to everything was "big."

The lesson generalizes: in a system where the learning signal drives *structural* change, a sign error doesn't just slow learning down, it reshapes the topology into something pathological. The blast radius of a bad signal is larger here than in a standard net, because the signal isn't only moving weights, it's moving *structure*.

6. Regions are the part I think is undervalued

A homogeneous self-organizing network collapses. I'm fairly confident of this now, both from reasoning and from watching it happen. If every part of the graph has the same plasticity, the same decay, the same membrane strictness, then either everything is plastic (and the structure drifts catastrophically, never stabilizing) or everything is locked (and it can't learn). There is no single global setting that gives you a stable-but-adaptable system.

Biology's answer is that different tissues have different rules. Sensory cortices are wildly plastic during critical periods and mostly locked afterward. Association cortex stays plastic for life. Hippocampus is rapid-plasticity, high-turnover. Basal ganglia learn by reinforcement. The rules are *regional*.

So the HNN has **regions**: containment subtrees, each with its own configuration object covering membrane strictness, portal-topology constraints, learning rate, energy decay rate, modulator sensitivities, and whether it accepts external input. A region isn't a label; it's a config the dynamics engine consults every time it touches a holon inside that region.

A region is a config object the dynamics engine consults on every touch. The whole differential-memory behavior below comes out of one field (`decay_rate`) differing between two of them:

```
RegionConfig:
    decay_rate           : float   # energy lost per tick
    membrane_strictness  : float   # how readily inputs are rejected
    learning_rate        : float   # interior + portal plasticity
    reward_sensitivity   : float   # how strongly modulators bite here
    accepts_input        : bool    # is this a sensory entry region

REGIONS = {
    "entry":       RegionConfig(decay_rate=0.30, membrane=0.2, lr=0.10, accepts_input=True),
    "semantic":    RegionConfig(decay_rate=0.005, membrane=0.8, lr=0.02),  # durable facts
    "episodic":    RegionConfig(decay_rate=0.15,  membrane=0.5, lr=0.08),  # events fade
    "preferences": RegionConfig(decay_rate=0.05,  membrane=0.6, lr=0.05, reward_sensitivity=0.9),
    "integrator":  RegionConfig(decay_rate=0.10,  membrane=0.3, lr=0.04),  # loose, heterogeneous
}
```

The load-bearing numbers are the two decay rates: semantic at `0.005` per tick, episodic at `0.15`. That is a 30× difference in how fast a memory cools, and it is the *entire* mechanism behind what follows.

The single most satisfying thing the whole system did was the interaction between the episodic and semantic regions. Feed it facts and events, let it idle, and the episodic energy decays away while the semantic energy persists: a ~24× energy ratio after a few dozen idle ticks, from nothing but the difference in decay rates. The system "remembered" the durable stuff and "forgot" the ephemeral stuff, and I didn't write a single line of code that said "forget the episode." It fell out of the regional physics. That's the moment the framework felt real to me.

This is also, not coincidentally, the least novel part; differential decay across memory systems is textbook. But having it emerge from a config difference rather than a special-cased forgetting routine is the thing I'd point to if someone asked "what does this architecture actually buy you."

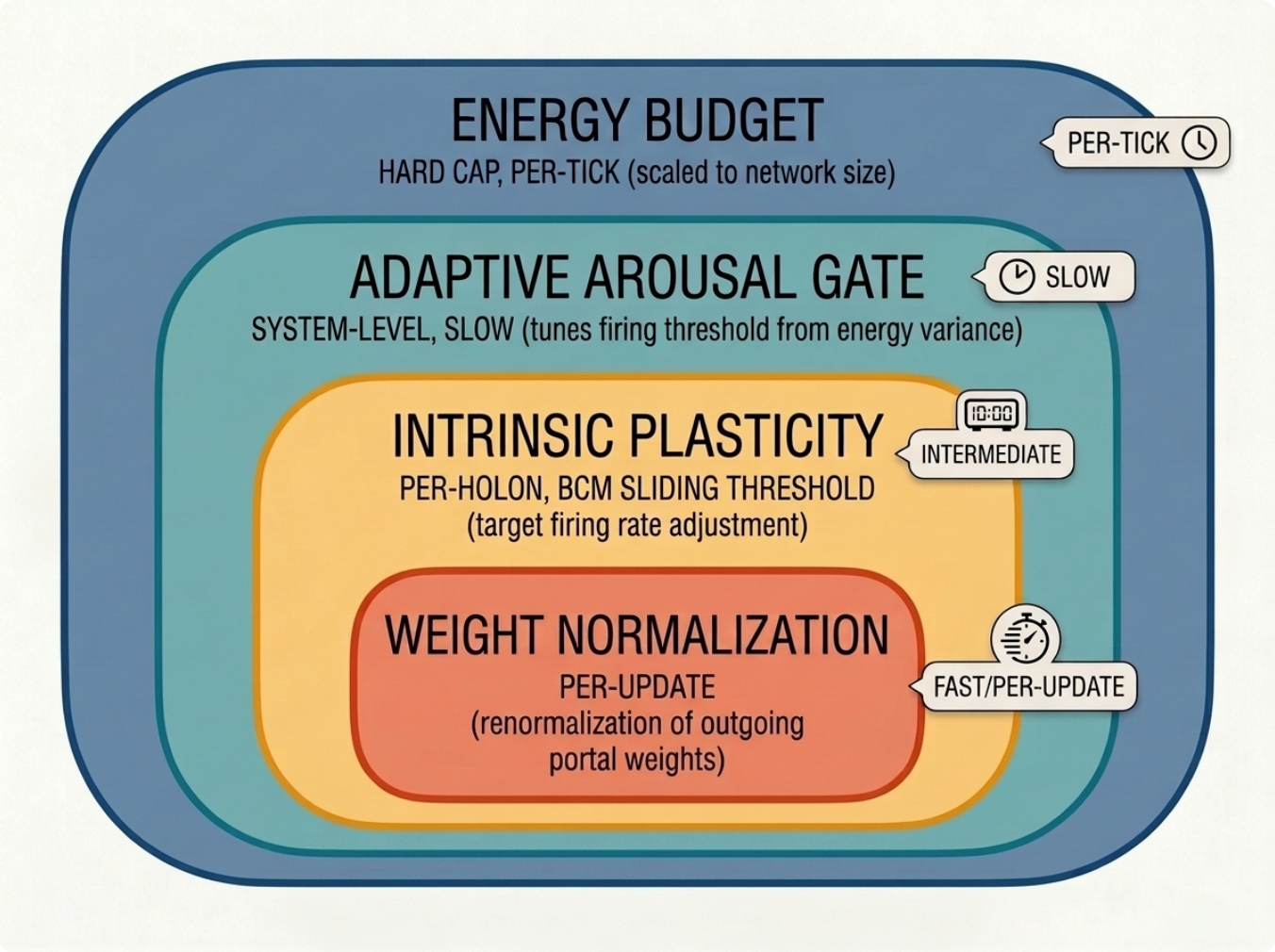


Figure 3. Two decay curves from the same starting energy. The slow-decay semantic region (~0.005/tick) barely declines while the fast-decay episodic region (~0.15/tick) fades to near zero, a ~24x gap after a few dozen idle ticks, with no explicit forgetting routine.

---

7. Where my experiment eventually fell on its face

Now for the reason I claim my attempt was mostly a failure.

The HNN has three learning mechanisms operating at different levels, and they don't cohere as well as I wanted.

Portal weight learning is the reward-modulated local reinforcement from Neuromodulation. This works. It's local, online, compatible with changing topology. It's the healthiest part of the learning story.

Interior learning is whatever updates a holon's internal state: triple addition in a symbolic interior, gradient descent on an embedding in a tensor interior. This also basically works, because inside a single holon you can use whatever mechanism you like, including plain backprop, since the interior is small and stable.

Structural learning (holons splitting when overloaded, merging when cold and similar, portals forming between co-active holons, regions crystallizing) is the distinctive idea and the one that is not, in any real sense, solved. These operations are triggered by the energy landscape and modulator state, not by gradients:

```
def structural_step(holarchy):
    for h in holarchy.holons:
        if h.interior.size > SPLIT_MAX or h.reject_rate > REJECT_MAX:
            split(h)                      # overloaded -> two holons + a portal
    for a, b in cold_similar_pairs(holarchy):
        if a.energy < COLD and b.energy < COLD and sim(a, b) > MERGE_SIM:
            merge(a, b)                   # two cold, similar holons -> one
    for a, b in coactive_pairs(holarchy):
        if coactivation(a, b) > FORM_THRESH and not linked(a, b):
            form_portal(a, b)             # consistent co-firing -> new portal
    for p in holarchy.portals:
        if p.traversals < PRUNE_MIN and p.weight < PRUNE_W:
            prune(p)                      # cold, weak -> removed (provenance kept)
```

They're closer to developmental or evolutionary mechanisms than to gradient descent. And the problem is stability: I could not find a setting of those six thresholds (`SPLIT_MAX`, `MERGE_SIM`, `FORM_THRESH`, `PRUNE_MIN`, and the rest) that let the structure adapt without either thrashing (endless split/merge cycles on the same holons) or ossifying (nothing ever changes). The window between "too eager" and "too timid" was narrow and scale-dependent and I never found a principled way to set it.

The deepest issue is credit assignment. Backprop's whole magic is that it assigns credit across an arbitrarily deep differentiable path. The HNN gives that up on purpose (you can't backprop through a topology that's rearranging itself, and you can't backprop through a SPARQL query). So credit assignment is *local*: the portal that fired gets the reward. But local credit assignment cannot learn deep compositional structure. It can't discover that a good outcome required a specific four-hop path through the graph and apportion credit along it. In practice this showed up as a hard ceiling on compositional generalization; the network could memorize associations but could not learn a *rule* that composed them. On a task where it had to combine two learned facts into an unseen combination, it scored at chance. A vanilla two-layer network would beat it. That's not a tuning problem; it's a consequence of the substrate's core commitment.

I tried to compensate with homeostatic mechanisms, and this is the one place the "just needs engineering" story is partly true.

7.1 The homeostatic patch, and why it half-worked

Every failure in Energy and Neuromodulation was, at bottom, an instability that I had to notice and fix by hand. That's damning for a system that's supposed to be self-organizing. Biology doesn't have an engineer watching the arousal gate. So I went looking for the mechanisms biology uses to keep itself stable without supervision, and implemented four of them:

- **Intrinsic plasticity (a BCM-style sliding threshold).** Each holon tracks its own recent firing rate and adjusts its activation threshold toward a target rate. Fire too much, threshold rises, you become selective. Fire too little, threshold drops, you become sensitive. This is the Bienenstock–Cooper–Munro rule (1982), lifted to the holon level. It's real neuroscience and it maps cleanly.
- **Synaptic scaling (weight normalization).** After learning, a holon's outgoing portal weights are renormalized to a target sum, preserving their *relative* pattern while bounding the total. This is Turrigiano's

synaptic scaling. It's what stops one portal from eating the entire signal budget (the pathology behind the "always predicts big" bug).

- **An energy budget.** Total system energy is capped and scaled to network size; if the network exceeds it, everything scales down proportionally. Metabolic constraint as a hard ceiling. This is what finally killed the "energy can't decay" failure mode structurally rather than by hand-tuning.
- **An adaptive arousal gate.** The gate coefficient that burned me in Energy now tunes itself based on the variance of the energy distribution: high variance (a few holons hogging energy) pushes the gate up, low variance relaxes it.

Stacked, they run at the end of every tick, each on its own timescale (normalization per update, plasticity and budget per tick, the gate slowly across many ticks):

```
def homeostasis(holarchy):
    # (a) per-update: renormalize each holon's outgoing weights to a target sum
    for h in holarchy.holons:
        total = sum(p.weight for p in h.portals)
        if total > 0:
            for p in h.portals:
                p.weight *= (TARGET_SUM / total)      # keep pattern, bound total

    # (b) per-holon, per-tick: BCM sliding threshold toward a target firing rate
    for h in holarchy.holons:
        h.firing_rate = 0.9 * h.firing_rate + 0.1 * (h.energy > threshold(h))
        h.threshold  += THETA_LR * (h.firing_rate - TARGET_RATE)

    # (c) system, per-tick: hard metabolic cap, scaled to network size
    budget = ENERGY_PER_HOLON * len(holarchy.holons)
    total  = sum(h.energy for h in holarchy.holons)
    if total > budget:
        for h in holarchy.holons:
            h.energy *= (budget / total)

    # (d) system, slow: adapt the arousal gate to energy variance
    v = variance(h.energy for h in holarchy.holons)
    global GATE
    GATE += GATE_LR * (v - TARGET_VARIANCE)           # volatile -> gate up
```

Read the parameter list in that block (`TARGET_SUM`, `THETA_LR`, `TARGET_RATE`, `ENERGY_PER_HOLON`, `GATE_LR`, `TARGET_VARIANCE`) and you can already see the problem I'm about to describe.

Here's my assessment of the patch. Individually, each mechanism is well-grounded and each fixed the specific failure it targeted. Collectively, they revealed the real problem: **I was adding feedback loops to stabilize the instabilities created by other feedback loops.** Each new homeostatic mechanism is itself parameterized (target firing rate, budget per holon, target variance, adaptation rate) and interacts with the others. Biology got to tune this web over hundreds of millions of years of selection. I got a few hundred ticks of testing. At some point

the stabilizers become a source of complexity on par with the thing they're stabilizing, and you have to ask whether the architecture is buying its stability or borrowing it. I genuinely don't know where that line is, and finding it is one of the open problems I'd hand to someone smarter.

And critically: even a perfectly stable HNN would still have the credit-assignment ceiling. Homeostasis keeps the system alive; it doesn't make it learn deep structure. Those are different problems and I only made real progress on the first.

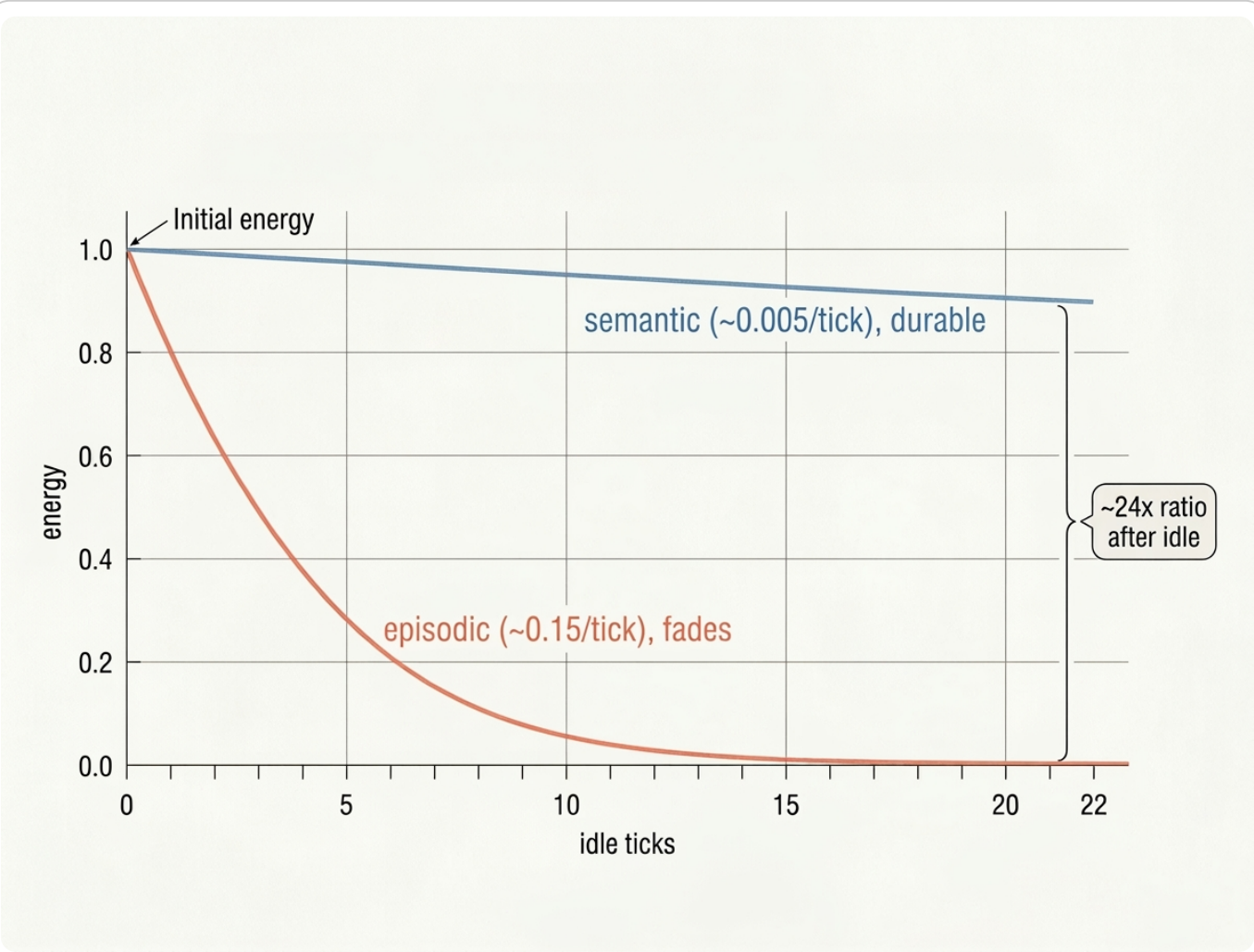


Figure 4. The four nested homeostatic control loops, each running on its own timescale: a per-tick energy budget, a slow system-level arousal gate, per-holon intrinsic plasticity, and per-update weight normalization at the core.

8. The section I got wrong and fixed about old brain and new brain

My first draft had a whole section built on the idea that the HNN should have an "old brain" (fast, affective, homeostatic, evolutionarily ancient) layered under a "new brain" (slow, symbolic, deliberative, evolutionarily recent), with the old modulating the new from below and the new regulating the old, slowly, from above. It leaned explicitly on the **triune brain** model.

That model is wrong. It isn't just contested; comparative neuroscientists reject it outright. Vertebrate brains share a common basic plan; reptiles have homologues of "limbic" structures; the neocortex is not a fresh layer bolted on top of a preserved reptilian core; and brain evolution does not proceed by stacking intact new modules over untouched old ones. As Cesario, Johnson, and Eisthen put it in a 2020 paper whose title I love (*"Your Brain Is Not an Onion With a Tiny Reptile Inside"*), the triune view "has long been discredited among neurobiologists." I had absorbed a pop-neuroscience myth and built a section on it.

Here's the thing, though: the *functional* intuition underneath survives the correction, if you re-ground it. The useful idea was never "there's an ancient reptile module." The useful idea is that **affect and homeostatic regulation ground and bias cognition**, that a system needs something like drives and something like an internal-state budget for its "reasoning" to be about anything. The right framing for that isn't triune layering; it's **allostasis** and **interoception** (the brain as an organ that regulates a body budget, in Lisa Feldman Barrett's framing, with affect as the felt sense of that budget rather than a separate ancient subsystem). There's no anatomical basement of oldness. There's a whole brain that is always, everywhere, regulating a body and constructing affect as part of how it does everything else.

Translated back to the HNN, the correction is clean and actually *simpler*: I don't need a segregated "old-brain region." I need **homeostatic holons** that produce the modulator fields (Neuromodulation) and are wired to matter (a stress signal that really does tighten membranes everywhere, a reward signal that really does gate plasticity), distributed through the holarchy rather than quarantined in a basement. The modulators *are* the affective grounding. I already had the mechanism; I just had the anatomy story wrong. Drop the reptile, keep the body budget.

I'm including this section rather than quietly deleting the mistake because the mistake is instructive: it's very easy, when you're reasoning by biological analogy, to reach for the version of the biology that's vivid and memorable and thirty years out of date. The analogy is a source of hypotheses, not evidence. Every biological claim in a framework like this needs to be checked against what neuroscience currently thinks, not against what you absorbed from a documentary.

---

9. World models, briefly, because it's where this wants to go

The most defensible long-term case for a substrate like this is world models. A world model is a structured, queryable internal simulation of an environment, something you can ask "what happens if I do X?" without running your entire network end to end. LeCun has made the world-model-centric case for autonomous intelligence (the JEPA line of work predicts in a learned representation space rather than in pixels); Ha and Schmidhuber showed years ago that an agent can learn to plan "inside a dream" of a learned world model.

The reason the HNN substrate is a natural host: a world model wants exactly the properties the holon already has. Structured, inspectable interiors (a holon holds a fragment of the model as queryable state). Governed portals as causal or temporal links (a CONSTRUCT that turns a state into its predicted successor is a little dynamics model). Containment as abstraction levels (perceptual predictions low, object and event models in the middle, agent and narrative models on top). Provenance as a counterfactual substrate ("what if this portal had fired differently?" is a query against a modified history).

The disagreement about whether LLMs already contain world models becomes tractable in this framing: an implicit model you can only access by generating text is not a *queryable* model. The Othello-GPT results suggest transformers can learn causally-relevant internal state; other work argues they fail to recover the true underlying rules. Both can be true. The HNN's contribution to that debate is to insist that structural accessibility (being able to ask the model about an entity or a counterfactual without running the whole thing forward) is the property that matters, and to make that property cheap by construction.

I did not get far enough to test any of this. It's the part of the paper that's still purely a bet. But it's the bet I'd most want someone to take up, because it's where the "structure is the computation" commitment would actually earn its keep: a world model is exactly the kind of thing you want to be able to inspect, query, and modify structurally, and that is the one thing this substrate is unambiguously good at.

---

10. What I'd tell the next person

Let me separate the ledger cleanly, because "mostly failed" is a summary, not a verdict, and the parts deserve different verdicts.

What worked, and I'd defend:

- **Structure and computation as one object.** Having the network's topology *be* a queryable graph, inspectable at every moment via SPARQL, was an unambiguous win for development, debugging, and analysis. When something went wrong, I could literally ask the structure what state it was in, mid-run, with no instrumentation. You cannot do that with a weight tensor. This alone I think is underexploited by the field. A live query against the running network looked like:

```
# "which holons are hot, and in which region?"
# a live query against the running network.
# try writing this against a weight tensor.
SELECT ?holon ?region ?energy WHERE {
    ?holon a hnn:Holon ;
           hnn:energy ?energy ;
           hnn:memberOf ?region .
    FILTER (?energy > 0.5)
} ORDER BY DESC(?energy)
```

- **Regional configuration.** Differential decay producing differential memory, with forgetting as a physical consequence rather than a coded routine, is the cleanest thing the framework does. The idea that a self-organizing net *needs* heterogeneous regional rules to avoid collapse is, I'm now convinced, correct.
- **Neuromodulation as produced global state.** Once the reward-sign bug was dead, reward-modulated local reinforcement behaved well and felt right. The three-factor grounding is solid.
- **Membranes as first-class rejection.** A connection that can *refuse* a signal rather than just attenuate it turns out to be a genuinely different and useful primitive, and it's the natural place for contradiction detection to live.

What failed, and why:

- **Structural learning is not solved.** The split/merge/form/prune operations never found a stable operating point. This is the core distinctive claim and it's the one I can't currently back up.
- **Local credit assignment caps compositional learning.** This is close to fundamental. Giving up backprop across the graph is what makes the topology mutable, but it's also what prevents deep compositional credit assignment. You may not get both. I don't have a full theory yet for that trade-off.
- **The dynamics are finicky in a way that undermines the self-organizing pitch.** Every stabilizing constant had to be found by hand, and the homeostatic mechanisms that automate that stabilization become their own tangle. A system that needs this much supervision to stay stable is not yet self-organizing in the sense that matters.

What I'm unsure about:

- Whether the whole thing scales. Everything I built topped out around a few hundred holons. The tick loop is roughly O(holons × portals) and the structural bookkeeping is worse. I have no evidence it survives contact with the scale where the interesting behavior (real regional specialization, meaningful modulator dynamics) is supposed to appear. It's entirely possible the framework only *becomes* itself at a scale where it also becomes computationally infeasible, and that would be a fatal irony.

If you're going to pick this up, my advice is: **don't start from the learning story.** Start from the structure. The neural-structural substrate (an inspectable graph that is also the computation, with typed governed connections and regional physics) is worth something even if you drive it with a completely conventional learning method, or with no learning at all and just hand-authored structure plus energy dynamics. Prove out the substrate as a *memory and reasoning surface* first. The self-organizing, self-learning holarchy is the dream, but it's downstream of a lot of unsolved problems, and the substrate has value before you get there. I think I inverted that order, spent my effort trying to make the dream learn, and under-invested in the part that was already working.

---

11. Why I still think the idea is right

Here is the primary thing I think is worth someone's attention.

Deep learning made a set of substrate commitments (fixed topology, scalar connections, homogeneous message passing, global differentiability, no modulatory state, no legible structure) and then spent a decade discovering that many of the hard remaining problems (continual learning, interpretability, compositional generalization, grounding, controllability) are downstream of exactly those commitments. We keep building elaborate machinery *on top of* the substrate to recover properties the substrate threw away: interpretability tools to make the opaque legible, continual-learning tricks to stop catastrophic forgetting, RAG to bolt an external memory onto a system with no native one.

The HNN is a bet that some of those problems are better addressed by *changing the substrate* than by patching over it. Make the structure legible by making it a graph. Make forgetting native by making it physical. Make memory native by making the network *be* the memory. Make connections governable by giving them contracts.

It's entirely possible this bet is wrong, that the substrate deep learning chose is chosen for good reasons (differentiability and dense tensor math are *stupendously* efficient on the hardware we have), and that the right move is to keep patching. My posterior after building the thing is maybe 60/40 against the HNN substrate as I designed it.

But the 40 is real, and I believe it's worth more attention: I think we've under-explored architectures where structure and computation are the same object, and I think we've done so partly because the dominant substrate makes that unification awkward, not because the unification is a bad idea. The holon is a good primitive. RDF makes it inspectable. Neuromodulation gives it global context. Regions give it stable heterogeneity. Those four things fit together more naturally than they have any right to, and that fit is the thing I couldn't talk myself out of even as the learning story fell apart.

I tried it. It mostly didn't work. I'm fairly sure the reasons it didn't work are more about my execution and the unsolved credit-assignment problem than about the core idea being unsound. And I'd rather publish the failure with the reasoning intact than let an idea I still believe in go unexamined.

A note on the code: I am not releasing the implementation repo. That is stubbornness more than principle. I want to keep tinkering with it for a while without scrutiny. If you are interested in the realization of the concept and want to talk about it, reach out to me on LinkedIn.

If you build a better version, tell me what broke.

Appendix of citations

The framework leans on a lot of real work. Where I've relied on something contested or non-peer-reviewed, I've said so.

- **Koestler, A.** *The Ghost in the Machine.* 1967. Origin of "holon" and "holarchy." (Note: also the source of the triune-brain framing I had to remove; see the section I got wrong and fixed.)
- **Van Brussel, H., Wyns, J., Valckenaers, P., Bongaerts, L., Peeters, P.** "Reference architecture for holonic manufacturing systems: PROSA." *Computers in Industry*, 1998. Holon as an engineering primitive.
- **Cagle, K.** Holonic Graph Architecture / four-graph holon model. Developed in public writing and the W3C Holon Community Group, 2025–26. *Emerging, not peer-reviewed.* Source of the four-graph decomposition and the SHACL-as-prediction-error / boundary-as-Markov-blanket framing.
- **W3C.** SHACL (Shapes Constraint Language), SPARQL, PROV-O, RDF specifications. Note SHACL validates single graphs; **Dao & Debruyne, SHACL-DS (2025)** addresses named-graph/dataset validation.
- **Kautz, H.** "The Third AI Summer." AAAI-2020 lecture. The six-type neurosymbolic taxonomy (*from a talk, never a formal paper*). For the peer-reviewed framing see **d'Avila Garcez, A. & Lamb, L.**, "Neurosymbolic AI: The 3rd Wave," *Artificial Intelligence Review*, 2023.
- **Frémaux, N. & Gerstner, W.** "Neuromodulated Spike-Timing-Dependent Plasticity, and Theory of Three-Factor Learning Rules." 2016. And **Gerstner et al.**, "Eligibility Traces and Plasticity on Behavioral Time Scales," 2018. Grounding for reward-modulated learning.

- **Bienenstock, E., Cooper, L., Munro, P.** (BCM theory), 1982. Sliding-threshold metaplasticity; the basis for the intrinsic-plasticity homeostatic mechanism.
- **Turrigiano, G. & Nelson, S.** Synaptic scaling / firing-rate homeostasis, 2004. Basis for the weight-normalization mechanism.
- **Friston, K.** Free-energy principle and Markov blankets (2006; 2010). *Influential but contested; the "too general to falsify" critique is live.* Basis for the boundary-as-Markov-blanket framing.
- **Barrett, L.F.** *Seven and a Half Lessons About the Brain* (2020); Barrett & Simmons, interoceptive predictive coding (2015). The allostasis/body-budget framing that replaced my triune-brain section.
- **Cesario, J., Johnson, D., Eisthen, H.** "Your Brain Is Not an Onion With a Tiny Reptile Inside." *Current Directions in Psychological Science*, 2020. Why the triune brain is discredited.
- **Nickel, M. & Kiela, D.** "Poincaré Embeddings for Learning Hierarchical Representations." NeurIPS 2017. Hyperbolic geometry for hierarchies; the natural embedding space for a holarchy.
- **Ha, D. & Schmidhuber, J.** "World Models." 2018. **LeCun, Y.** "A Path Towards Autonomous Machine Intelligence." 2022 (JEPA).
- **Shazeer et al.** (2017, sparse-gated MoE); **Fedus et al.** (Switch Transformer, 2022). MoE as the mainstream cousin of holonic specialization.
- **Kirkpatrick et al.** "Overcoming catastrophic forgetting in neural networks" (EWC), *PNAS* 2017. The continual-learning problem the HNN's native forgetting is meant to sidestep.

Dr. Zachary Welz